

11) Teniendo una red privada con direcciones IPv4 de clase A y sabiendo que:

- Se necesita reservar espacio para una subred /9.
- Se necesita una subred para un millón de hosts.
- Se necesita reservar espacio para 2 subredes cuyo tamaño de bloque es 65.536.
- Se necesitan 90 subredes punto a punto para conectar routers entre sí que contengan dirección de broadcast.

Calcular cuántas subredes adicionales de tamaño de bloque igual a 2048 se pueden obtener.

Tener en cuenta que existe:

- Clase A (126 redes, 16.777.216 hosts c/u) → 10.x.x.x /8
- Clase B (16.384 redes, 65.536 hosts c/u) → 172.x.x.x /16
- Clase C (2.097.152 redes, 256 hosts c/u) → 192.x.x.x /24

Arranco teniendo una red privada con direcciones IPv4 de clase A, osea

Obs: En el caso de que me den una red como por ejemplo “192.168.80.0/19” tengo que restar la mas cara de la misma forma: $2^{32-19} = 2^{13}$ direcciones

Direcciones totales hasta ahora: Clase A ⇒ 10.0.0.0 /8

⇒ $2^{32-8} = 2^{24}$ direcciones

Para el **1er ítem**: “Se necesita reservar espacio para una subred /9”

Osea que quieren un espacio de $2^{32-9} = 2^{23}$ direcciones.

Por ahora el cálculo va: $2^{24} - 2^{23}$ direcciones

Para el **2do ítem**: “Se necesita una subred para un millón de hosts.”

Osea que quieren: $\log_2(1.000.000) = 19,931 \approx 20 = 2^{20}$ direcciones.

Por ahora el cálculo va: $2^{24} - 2^{23} - 2^{20}$ direcciones

Para el **3er ítem**: “Se necesita reservar espacio para 2 subredes cuyo tamaño de bloque es 65.536”

Osea quieren $2 * 65536$ direcciones

Por ahora el cálculo va: $2^{24} - 2^{23} - 2^{20} - (2 * 65536)$ direcciones

Para el **4to ítem**: “Se necesitan 90 subredes punto a punto para conectar routers entre sí que contengan dirección de broadcast.”

La mínima con broadcast y 3 host es /30 $\Rightarrow 2^2 = 4$ direcciones por subred.

osea quieren $4 * 90$ direcciones

Por ahora el cálculo va: $2^{24} - 2^{23} - 2^{20} - (2 * 65536) - (4 * 90)$ direcciones

La pregunta final: Calcular cuántas subredes adicionales de tamaño de bloque igual a 2048 se pueden obtener.

$$\frac{2^{24} - 2^{23} - 2^{20} - (2 * 65536) - (4 * 90)}{2048} = 3519,824219 = \mathbf{3519}$$

DNS

12) A partir de la siguiente captura de pantalla:

```
; <<>> DiG 9.18.30-0ubuntu0.22.04.2-Ubuntu <<>> facebook.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 25153
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;facebook.com.                IN      A

;; ANSWER SECTION:
facebook.com.                52      IN      A      57.144.206.1

;; Query time: 10 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Wed Sep 10 19:14:48 -03 2025
;; MSG SIZE rcvd: 57
```

Conteste las siguientes preguntas o diga si son verdaderas (V) o falsas (F):

a. Ingrese el comando utilizado junto con sus argumentos:

dig facebook.com

b. ☐ La respuesta contiene una IPv4.

Verdadero. En la **ANSWER SECTION** figura un registro **A** (tipo A = IPv4):

facebook.com. 52 IN A 57.144.206.1

- **A** $\Rightarrow$ dirección IPv4.
- **57.144.206.1** tiene formato IPv4 (cuatro octetos decimales).
- Además, el **status: NOERROR** indica que la consulta fue resuelta correctamente y **ANSWER: 1** confirma que hay una respuesta.

c. ☐ El servidor no cuenta con una IPv6.

Falso (o “No se puede concluir con esta salida”).

La línea **SERVER: 127.0.0.53#53 (UDP)** solo te dice **qué resolver usaste y por qué dirección lo contactaste** (en este caso, **IPv4 loopback** del stub local).

De **ninguna** manera implica que ese servidor **no tenga** IPv6; simplemente esta conexión usó IPv4. Para afirmar algo sobre IPv6 deberías:

- Consultar al servidor por su **dirección IPv6** (si tiene) con **@** y **transportar por IPv6** (**dig -6**), o
- Pedir **registros AAAA** del nombre objetivo: **dig facebook.com AAAA**.

La salida **no muestra** si el resolver u “origin server” poseen IPv6; solo que esta consulta puntual fue por UDP/IPv4 al 127.0.0.53.

“usar el comando “dig” con las opciones iterativa, autorizada y verborrágica”

Modo iterativo significa que vos mismo (el cliente) vas siguiendo **la cadena de delegaciones DNS**, igual que lo haría un resolver recursivo, pero **paso a paso**.

```
dig +trace facebook.com
```

Empieza preguntando a los **servidores raíz** (“.”) → te responden los NS de **.com**.

Luego consulta a un **TLD server (.com)** → le devuelve los NS de **facebook.com**.

Por último pregunta a un **autoritative server** de **facebook.com** → obtiene la respuesta final.

Algo así:

```
.                518400 IN NS a.root-servers.net.
com.             172800 IN NS a.gtld-servers.net.
facebook.com.    86400  IN NS a.ns.facebook.com.
www.facebook.com. 300    IN A 157.240.3.35
```

Autorizada se refiere a hacer la consulta directamente a un **servidor autoritativo**, es decir, **el que tiene la zona con la información “oficial”** (no cacheada).

Para hacerlo necesitás saber **qué servidor NS es autoritativo**.

Ejemplo:

```
dig facebook.com NS +short
```

Te devuelve algo como:

```
a.ns.facebook.com.
b.ns.facebook.com.
c.ns.facebook.com.
```

Luego consultas a alguno de ellos directamente **sin recursión**:

```
dig @c.ns.facebook.com facebook.com A +norecurse
```

Salida:

```
; <<>> DiG 9.18.30-0ubuntu0.24.04.2-Ubuntu <<>> @c.ns.facebook.com
facebook.com A +norecurse
; (2 servers found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 22890
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;facebook.com.                IN      A

;; ANSWER SECTION:
facebook.com.                60      IN      A      57.144.206.1

;; Query time: 20 msec
;; SERVER: 2a03:2880:f1fc:c:face:b00c:0:35#53(c.ns.facebook.com) (UDP)
;; WHEN: Wed Oct 29 09:36:46 -03 2025
;; MSG SIZE rcvd: 57
```

¿Es una respuesta recursiva o iterativa?

→ Iterativa, porque usaste **+norecurse** y el flag **aa** confirma que viene del autoritativo.

¿Qué significa el flag **aa**?

→ “Authoritative Answer”: el servidor tiene autoridad sobre la zona consultada.

¿Cuál es el tipo de registro?

→ **A** → dirección IPv4.

¿Cuál es el TTL y qué implica?

→ 60 segundos → el resolver puede cachear la respuesta durante ese tiempo.

¿Qué protocolo y transporte se usó?

→ DNS clásico sobre UDP, versión EDNS(0).

¿Por qué la dirección del servidor empieza con **2a03**?

→ Porque es IPv6 → el cliente contactó al NS por IPv6, pero pidió un registro A (IPv4).

Verborrágico significa mostrar **toda la información detallada** del proceso de resolución, en lugar de solo el resultado.

Por defecto, **dig** ya es bastante detallado, pero podés combinar opciones para hacerlo **más “verbose” o explícito**:

- Mostrar todas las secciones, sin filtros (default).

- Para mayor control, se puede usar:

```
dig facebook.com +noall +answer +stats
```

(menos verborrágico)

o al revés:

```
dig facebook.com +all
```

(máximo detalle posible) Esta sería la opción si me piden verborrágico. Aunque por default, ya se muestra lo mismo en este caso.

Si me pidieran que **combine las tres ideas**, la más completa sería:

```
dig +trace facebook.com +all
```

o si ya se el NS:

```
dig @a.ns.facebook.com facebook.com A +norecurse +all
```

Objetivo	Opción dig	Qué muestra / sirve para
Ver cómo se resuelve paso a paso	+trace	Resolución iterativa completa (root → TLD → autoritativo)
Consultar directamente al autoritativo	@<ns> + +norecurse	Respuesta oficial (flag aa)
Ver todo el detalle	+all	Verborrágico (headers, flags, EDNS, etc.)
Ver solo el resultado limpio	+short	Solo la IP (útil para scripting, pero no para exámenes)
Consultar un tipo específico	<tipo> (A, AAAA, MX, NS, etc.)	Filtra el tipo de registro
Forzar IPv4 o IPv6	-4 o -6	Controla transporte
Ver DNSSEC	+dnssec	Agrega RRSIG y bits AD/CD
Consultar el autoritativo de una zona	+nssearch	Devuelve SOA y NS oficiales

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 22890
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
```

Campo	Significado
opcode	Tipo de operación. QUERY es una consulta normal. Otros: UPDATE , NOTIFY , etc.
status	Resultado de la consulta. 👉 NOERROR (ok), NXDOMAIN (el nombre no existe), SERVFAIL (fallo de servidor), REFUSED , FORMERR .
id	Número de identificación del mensaje DNS (sirve para emparejar consulta ↔ respuesta).
flags	Conjunto de indicadores de estado (ver tabla abajo).
QUERY / ANSWER / AUTHORITY / ADDITIONAL	Número de registros en cada sección del mensaje DNS.

```
;; QUESTION SECTION:
;facebook.com.          IN      A
```

Campo	Significado
facebook.com.	Nombre del dominio consultado.
IN	Clase “Internet” (casi siempre IN).
A	Tipo de registro: dirección IPv4.

1. Modos de uso comunes de dig

Campo / Comando	Descripción
Recursivo	dig facebook.com — consulta estándar al resolver local.
Iterativo	dig +trace facebook.com — sigue la cadena root → TLD → autoritativo.
Autoritativo	dig @a.ns.facebook.com facebook.com A +norecurse — pregunta directa al servidor oficial
Verbose / Verborrágico	dig facebook.com +all — muestra toda la información.
Simple / Limpio	dig facebook.com +short — devuelve solo la IP.
Reversa (PTR)	dig -x 8.8.8.8 — resuelve IP → nombre de dominio.

2. Flags principales del HEADER

Campo / Comando	Descripción
qr	Query/Response — indica que es una respuesta.
aa	Authoritative Answer — el servidor es dueño de la zona.
rd	Recursion Desired — el cliente pidió recursión.
ra	Recursion Available — el servidor puede realizar recursión.
ad	Authenticated Data — datos validados con DNSSEC.
cd	Checking Disabled — validación DNSSEC deshabilitada.

Tipos de consultas posibles con **dig**

Tipo de consulta	Qué hace	Ejemplo
Recursiva	El resolver hace todo el trabajo (default).	<code>dig facebook.com</code>
Iterativa	Tu cliente va saltando root → TLD → autoritativo.	<code>dig +trace facebook.com</code>
Autoritativa	Consultas directo a un NS oficial sin recursión.	<code>dig @a.ns.facebook.com facebook.com A +norecurse</code>
Verbose / Verborrágica	Muestra toda la información (sin <code>+short</code>).	<code>dig facebook.com +all</code>
Limpia / simple	Solo el resultado.	<code>dig facebook.com +short</code>
DNSSEC	Solicita datos firmados.	<code>dig facebook.com +dnssec</code>
Reversa (PTR)	Convierte IP → nombre.	<code>dig -x 8.8.8.8</code>

3. Estructura de salida típica

Campo / Comando	Descripción
HEADER	Contiene opcode, status, flags y conteo de secciones.
QUESTION	Indica qué se preguntó (nombre, clase, tipo).
ANSWER	Contiene la respuesta principal (nombre, TTL, clase, tipo, valor).
AUTHORITY	Indica servidores con autoridad sobre la zona.
ADDITIONAL	Muestra datos adicionales (ej. IPs de los NS).
OPT PSEUDOSECTION	Metadatos de EDNS, como tamaño UDP o DNSSEC.

4. Tipos de registros DNS más comunes

Campo / Comando	Descripción
A	Dirección IPv4 del host.
AAAA	Dirección IPv6 del host.
CNAME	Alias (canonical name).
MX	Servidor de correo.
NS	Name Server de la zona.
SOA	Start of Authority, datos base de la zona.
PTR	Entrada de búsqueda inversa (IP → nombre).
TXT	Texto arbitrario, usado para SPF, verificación, etc.

5. Ejemplos interpretados

Campo / Comando	Descripción
Respuesta autoritativa	flags: qr aa — Responde un servidor que administra la zona.
Respuesta recursiva	flags: qr rd ra — Respuesta cacheada o del resolver local.
Iterativa	dig +trace facebook.com — muestra la ruta desde root hasta el NS autoritativo.
NXDOMAIN	El dominio consultado no existe.
TTL	Indica por cuánto tiempo la respuesta puede ser cacheada.

Si ves **rd ra** → consulta **recursiva**, respuesta de un **resolver**.

Si ves **aa** sin **ra** → **respuesta autoritativa** (usaste **+norecurse**).

status: NXDOMAIN → el dominio **no existe**.

+trace → siempre iterativa, muestra **todas las delegaciones**.

A → IPv4, **AAAA** → IPv6.

SERVER: 127.0.0.53 → resolución hecha por el **resolver local del sistema**.

TTL bajo → sitios con **balanceo de carga dinámico** (Facebook, Google, etc.).

EDNS indica uso de extensiones modernas (UDP >512 bytes).

Herramientas de software

13) A partir de la siguiente captura de pantalla, sabiendo que el dispositivo 9 es el destino:

```
1 192.168.1.1 4.201 ms 4.103 ms 4.709 ms
2 181.89.6.100 11.754 ms 11.733 ms 11.713 ms
3 * * *
4 * * *
5 181.89.51.39 11.902 ms * 14.845 ms
6 72.14.194.198 14.821 ms 13.414 ms 13.346 ms
7 108.170.255.31 12.827 ms 192.178.73.87 10.116 ms 192.178.80.111 9.866 ms
8 209.85.250.153 10.037 ms 142.251.239.157 9.740 ms 142.251.239.155 9.767 ms
9 8.8.8.8 9.708 ms 10.403 ms 10.354 ms
```

Conteste las siguientes preguntas o diga si son verdaderas (V) o falsas (F):

a. Escriba el comando utilizado (no es necesario incluir flags):

tracert 8.8.8.8

se puede observar que es el destino porque es la última línea a la que llega.

- **tracert** se usa para determinar **la ruta que siguen los paquetes IP** desde el host origen hasta el destino.
- Por defecto, envía **tres paquetes por salto (TTL)**, incrementando el valor del TTL desde 1 hasta que llega al destino final. Cada Salto es el TTL con el que se manda. Por ejemplo:
Salto 1 → TTL = 1
Salto 2 → TTL = 2
....
Salto 9 → TTL = 9
Por esto es que se dice que el paquete tardó 9 saltos en llegar.
El comando se puede ejecutar con otra cantidad de paquetes de la siguiente forma:
-q <nro paquetes>
- Cada router intermedio reduce el TTL y, al llegar a 0, envía un **ICMP Time Exceeded** al origen.
Esto le permite al tracert saber qué router respondió y cuánto tardó cada paquete (latencias en ms).
Si figura "*" es porque el paquete expiró pero el router no envió el ICMP de vuelta (ya sea por política de firewall o saturación).
- En este caso, el destino (**8.8.8.8**) es el **dispositivo 9**, que pertenece a Google DNS.
- **Si aparecen dos líneas consecutivas con * * ***, ¿significa que el enlace está cortado?
⇒ No necesariamente. Puede significar que los routers intermedios no envían respuestas ICMP por política de seguridad, pero siguen reenviando el tráfico correctamente (la traza continúa).

- ¿Cuántas veces puede aparecer * * * seguidas?

⇒ No hay un límite fijo; depende de las políticas de los routers intermedios. Pueden aparecer una o varias líneas seguidas de * * * sin que exista un problema físico en la red, siempre que la traza se reanude en saltos posteriores.

- b. ☐ El host envió menos de diez paquetes.

FALSOOOOO. Traceroute envía **al menos tres paquetes por cada salto (por defecto)**.

Podemos ver que hay **9 saltos** (líneas numeradas del 1 al 9), y en casi todas aparecen tres tiempos (ms), lo que indica tres paquetes enviados.

9 saltos × 3 paquetes = **27 paquetes mínimos** enviados.

Además, incluso los saltos donde aparecen “*” significan **paquetes enviados pero sin respuesta**, no que no fueron enviados.

- c. ☐ El router 5 respondió con un paquete ICMP.

Aca depende el contexto como se tome la pregunta. Es **true** si se toma como “al menos un paquete ICMP” y es **false** si se toma como “un único paquete ICMP”, ya que como se ve, responde con un tiempo - * - un tiempo ⇒ hay dos respuestas de paquete **ICMP**.

- d. ☐ El router 3 y el router 4 están desconectados de la red eléctrica.

FALSO. Los * * * no significan que se corte el camino, sino que por seguridad, no responden con un mensaje **ICMP**.

- e. ☐ Paquetes con igual TTL tuvieron distintos caminos.

VERDADERO. Podemos observar que en algunas líneas aparecen múltiples direcciones IP distintas en el mismo salto, por ejemplo:

```
7 108.170.255.31 12.827 ms 192.178.73.87 10.116 ms 192.178.80.111 9.866 ms
8 209.85.250.153 10.037 ms 142.251.239.157 9.740 ms 142.251.239.155 9.767 ms
```

Esto significa que, en el mismo “salto” (es decir, con el mismo valor de TTL), **los tres paquetes siguieron rutas diferentes** y llegaron a **routers distintos**, aunque todos pertenecen a la misma red de Google. Esto es típico del **load balancing o balanceo de carga**.

Herramientas de software

13) A partir de la siguiente captura de pantalla:

```
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.1:45389         0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:631          0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53          0.0.0.0:*               LISTEN
tcp        0      0 192.168.1.2:58514      20.189.173.1:443        ESTABLISHED
tcp        0      0 192.168.1.2:33878      40.71.11.167:443        ESTABLISHED
tcp        0      0 192.168.1.2:35864      31.13.94.52:5222        ESTABLISHED
tcp        0      0 192.168.1.2:57996      4.228.31.150:22         TIME_WAIT
tcp        0      0 127.0.0.1:45389        127.0.0.1:44856         ESTABLISHED
tcp        0      0 127.0.0.1:45389        127.0.0.1:39310         ESTABLISHED
tcp        0      0 192.168.1.2:34556      172.64.148.235:443      ESTABLISHED
tcp        0      0 192.168.1.2:39284      34.107.243.93:443       ESTABLISHED
tcp        0      0 192.168.1.2:54744      31.13.94.52:443         ESTABLISHED
tcp        0      0 192.168.1.2:45008      104.18.32.47:443        TIME_WAIT
tcp        0      0 192.168.1.2:44300      31.13.94.52:443         ESTABLISHED
tcp        1      0 192.168.1.2:35882      89.222.120.6:443        CLOSE_WAIT
tcp        0      0 192.168.1.2:51030      13.107.246.33:443       ESTABLISHED
tcp        0      0 127.0.0.1:39310        127.0.0.1:45389         ESTABLISHED
tcp        0      0 192.168.1.2:54112      151.101.217.91:443      ESTABLISHED
tcp        0      1 192.168.1.2:44170      169.254.169.254:80      SYN_SENT
tcp        0      0 127.0.0.1:44856        127.0.0.1:45389         ESTABLISHED
tcp        0      0 192.168.1.2:40940      34.120.208.123:443      ESTABLISHED
tcp        0      0 192.168.1.2:36868      151.101.217.91:443      ESTABLISHED
tcp        0      0 192.168.1.2:36884      151.101.217.91:443      ESTABLISHED
tcp        1      0 192.168.1.2:41456      45.55.41.223:443        CLOSE_WAIT
tcp        0      0 192.168.1.2:54752      31.13.94.52:443         ESTABLISHED
tcp        0      0 192.168.1.2:39496      54.94.147.102:443       ESTABLISHED
tcp        0      0 192.168.1.2:58502      20.189.173.1:443        ESTABLISHED
tcp6       0      0 :::1:631               :::*                     LISTEN
udp        0      0 0.0.0.0:10420          0.0.0.0:*               ESTABLISHED
udp        0      0 127.0.0.53:53          0.0.0.0:*               ESTABLISHED
udp        0      0 192.168.1.2:68         192.168.1.1:67          ESTABLISHED
udp        0      0 0.0.0.0:41845          0.0.0.0:*               ESTABLISHED
udp        0      0 0.0.0.0:5353           0.0.0.0:*               ESTABLISHED
udp        0      0 0.0.0.0:42287          0.0.0.0:*               ESTABLISHED
udp6       0      0 :::49294               :::*                     ESTABLISHED
udp6       0      0 fe80::8664:b9a6:ca2:546  :::*                     ESTABLISHED
udp6       0      0 _:::5353               :::*                     ESTABLISHED
```

Conteste las siguientes preguntas o diga si son verdaderas (V) o falsas (F):

a. Escriba el comando utilizado (no es necesario incluir flags):

netstat

b. ☐ El protocolo de capa de transporte del socket cuyo puerto de origen es 49294 es UDP6.

FALSO. No existe UDP6 como protocolo de capa de transporte. UDP6 lo que indica es que es una familia de direcciones IPv6.

- c. ☐ El 3-way-handshake de la aplicación que creó el socket cuyo local address es 127.0.0.1:44856 fue efectuado con éxito.

VERDADERO. Figura como **ESTABLISHED**, buscamos:

```
tcp 0 0 127.0.0.1:45389 127.0.0.1:44856 ESTABLISHED
```

- Es una conexión **TCP** entre dos procesos locales (loopback IPv4).
- Estado **ESTABLISHED** significa que el **3-way-handshake** (SYN → SYN-ACK → ACK) **ya fue completado con éxito**.
- Ambos puertos pertenecen a la misma máquina (cliente y servidor en 127.0.0.1).

- d. ☐ No es posible tener dos sockets UDP, utilizando IPv4, escuchando en el mismo puerto.

FALSO. Dos sockets **pueden escuchar en el mismo puerto y dirección**. Lo vimos en una clase práctica.

UDP, al ser un protocolo **sin conexión** (connectionless), **no mantiene estado de sesión** como TCP.

Esto hace que **el kernel permita más flexibilidad** en cómo los sockets pueden compartir puertos.

En particular, lo que determina si dos sockets pueden coexistir en el mismo puerto depende de:

- La **dirección IP local** asociada al socket.
- Si se activan opciones especiales como **SO_REUSEADDR** o **SO_REUSEPORT**.

- e. ☐ El 3-way-handshake de la aplicación que creó el socket cuyo local address es 192.168.1.1:68 fue efectuado con éxito.

FALSO. La dirección 192.168.1.1:68 ni siquiera figura.

Lo que sí figura es este:

```
udp 0 0 192.168.1.2:68 192.168.1.1:67 ESTABLISHED
```

Lo cual es engañoso, ya que el protocolo es UDP y no TCP.

UDP **no utiliza handshake**, ya que no está orientado **a conexión**.

El campo "ESTABLISHED" en este contexto es engañoso: algunos sistemas lo muestran solo para indicar que hay comunicación activa, pero **no hay ningún intercambio de SYN/SYN-ACK/ACK**. Con lo cual, no hay **3-way-handshake**.

Estados TCP mas comunes

Estado	Descripción
LISTEN	Servidor esperando conexiones entrantes.
SYN_SENT	Cliente envió SYN, esperando respuesta.
SYN_RECV	Servidor recibió SYN, envió SYN-ACK.
ESTABLISHED	Conexión activa (3-way-handshake completado).

TIME_WAIT	Espera antes de cerrar completamente la conexión.
CLOSE_WAIT	El otro extremo cerró, este espera cierre local.
FIN_WAIT1 / 2	Fase de finalización TCP.

- “¿Cuál es la diferencia entre **LISTEN** y **ESTABLISHED**?”
 ⇒ **LISTEN**: socket del lado servidor esperando conexiones (pasivo).
ESTABLISHED: conexión TCP activa, handshake completado (bidireccional).
- “¿Existe algún otro estado que indique un 3-way-handshake exitoso que no sea **ESTABLISHED**?”
 Si la conexión **está ahora** en **FIN_WAIT_1**, **CLOSE_WAIT**, **TIME_WAIT**, o **LAST_ACK**
 → **sí se puede afirmar que hubo un 3-way handshake exitoso**,
 porque esos estados **solo existen después** de haber estado en **ESTABLISHED**.
 PRESTAR ATENCIÓN A QUE EL PROTOCOLO SEA TCP.

- 14) A partir de la siguiente captura de pantalla del análisis de un paquete utilizando Wireshark, el cual es parte del envío de una imagen de un gato utilizando un sombrero desde un servidor a un cliente:

```
Frame 11: 76 bytes on wire (608 bits), 76 bytes captured (608 bits) on interface any, id 0
Linux cooked capture v1
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 60
  Identification: 0x0000 (0)
  Flags: 0x40, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: TCP (6)
  Header Checksum: 0x3cba [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 127.0.0.1
  Destination Address: 127.0.0.1
Transmission Control Protocol, Src Port: 9500, Dst Port: 51808, Seq: 0, Ack: 1, Len: 0
  Source Port: 9500
  Destination Port: 51808
  [Stream index: 2]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 211560415
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 115590082
  1010 .... = Header Length: 40 bytes (10)
  Flags: 0x012 (SYN, ACK)
  Window: 65483
  [Calculated window size: 65483]
  Checksum: 0xfe30 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
  [Timestamps]
  [SEQ/ACK analysis]
```

Conteste las siguientes preguntas o diga si son verdaderas (V) o falsas (F):

- a. Escriba el puerto en el que escucha el servidor:

- b. ☐ El emisor envió 1 Byte de contenido de capa de aplicación.
- c. ☐ La comunicación transcurre en una red local privada.
- d. ☐ El paquete capturado no contiene información de capa de aplicación, contiene únicamente los headers de TCP y de IPv4.

a)

El paquete pertenece a (la línea con color celeste claro):

Transmission Control Protocol, Src Port: 9500, Dst Port: 51808, Seq: 0, Ack: 1, Len: 0
Flags: 0x012 (SYN, ACK)

Y en la capa IP:

Source Address: 127.0.0.1

Destination Address: 127.0.0.1

Esto nos dice varias pistas:

- Dirección origen y destino: **127.0.0.1**, o sea comunicación **local (loopback)**.
- El **puerto origen 9500** → corresponde al **servidor**. (ver que dice "Src Port: 9500", osea Source Port = Puerto Origen).
- El **puerto destino 51808** → corresponde al **cliente**.

- Flags: **SYN, ACK** → el paquete forma parte del **3-way handshake** de TCP (segundo paso).
- Longitud: **Len: 0** → no hay datos de capa de aplicación (solo control).

El servidor escucha en el **puerto 9500**.

b)

FALSO. En la captura, se ve “Len: 0”. Si dijera “len:1” si sería verdadera ya que cuenta por bytes.

c)

FALSO. Porque **127.0.0.1** no pertenece a ninguna red local, sino al **rango de loopback**, que **solo existe dentro del propio equipo** y nunca se transmite físicamente.

Para el caso de **red local privada real (LAN)**:

- 10.0.0.0 – 10.255.255.255
- 172.16.0.0 – 172.31.255.255
- 192.168.0.0 – 192.168.255.255

Ejemplo real:

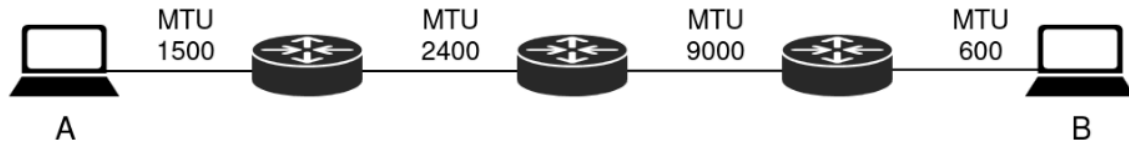
Internet Protocol Version 4, Src: 192.168.1.10, Dst: 192.168.1.25

Transmission Control Protocol, Src Port: 443, Dst Port: 51808, Seq: 123, Ack: 456, Len: 512

Esto significa que un host de tu LAN (por ejemplo, tu notebook) se está comunicando con otro host en la misma red (por ejemplo, una impresora o servidor local).

Fragmentación

El host A envía un paquete de 1601 bytes al host B, con la siguiente configuración de MTUs (en bytes). El header IPv4 no tiene opciones y no hay reordenamiento de fragmentos en la red.



- 1) Calcular la cantidad de fragmentos que llegan al host B.

A ⇒ R1:

MTU: 1500

⇒ $1601 \Rightarrow 1500 - 20 = 1480 \Rightarrow 1480 / 8 = 185$ OK

1) 1480 bytes

2) $1601 - 1480 = 121$ bytes

Nro de fragmento	Offset	Total Lenght	Payload Lenght	MF	DF
1)	0	1500	1480	1	0
2)	185	141	121	0	0

R1 ⇒ R2: todo ok

R2 ⇒ R3: todo ok

R3 ⇒ B:

MTU: 600

⇒ $600 - 20 = 580 \Rightarrow 580 / 8 = 72.5 = 72 \Rightarrow 72 * 8 = 576$

1) $1480 / 576 = 2,569 = 3$ fragmentos ⇒ $1480 - 576 - 576 = 328$

Nro de fragmento	Offset	Total Lenght	Payload Lenght	MF	DF
1.1)	0	596	576	3	0
1.2)	72	596	576	2	0
1.3)	144	348	328	1	0
2)	185	141	121	0	0

Respuesta ⇒ 4

2) Llenar la siguiente tabla con los campos del segundo fragmento que llega.

Nro de fragmento	Offset	Total Lenght	Payload Lenght	MF	DF
1.2)	72	596	576	2	0

3) Defina si las siguientes opciones son verdaderas (V) o falsas (F):

- a. ☐ En IPv4, el bit DF evita que un router fragmente el paquete pero no impide que el host origen lo haga.
- b. ☐ La fragmentación ocurre únicamente en los routers.
- c. ☐ En una topología real, todos los fragmentos recorren el mismo camino para llegar al destino.
- d. ☐ Si se pierde un fragmento IPv4, los demás fragmentos pueden ser entregados a la capa superior parcialmente.

a)